

EXPLORATORY ANVIL PROJECT

2×2 Ready/Valid Concurrent Packet Router

1. Introduction and Objectives

This project implements and verifies a 2-input, 2-output concurrent ready/valid packet router, in three parts done in order: (A) a SystemVerilog reference implementation and simulation, (B) an SVA specification and verification of that design, and (C) an AnvilHDL reimplementation checked against the SystemVerilog reference for equivalence.

The router carries 32-bit data with a 1-bit TDEST field (TDEST=0 → Output 0, TDEST=1 → Output 1), ready/valid handshaking, multi-beat packets terminated by TLAST, packet locking, and per-output round-robin arbitration. A transfer occurs only when VALID && READY. This design was chosen because it is the smallest system that still exercises all four properties the brief requires — concurrent ports, a genuine backpressured handshake, real contention that must be arbitrated, and multi-beat transfers that must not be corrupted or interleaved — while keeping the state space small enough to specify completely in SVA and port to Anvil in the time available.

Objectives: implement the router in SystemVerilog; verify it with SVA; reimplement it in AnvilHDL; generate and lint SystemVerilog RTL from the Anvil source; build an equivalence testbench comparing the two implementations across routing, contention, backpressure, packet locking, and round-robin arbitration; and document where SVA/SystemVerilog and AnvilHDL semantics diverge.

2. Part A — SystemVerilog Implementation and Verification

router.sv implements two input ports and two output ports. Each input generates a request for an output from its TDEST value (e.g. Input 0 requests Output 0 when in0_tvalid=1 and in0_tdest=0). Each output has independent round-robin arbitration: a pointer rr selects the priority input on contention (rr=0 favours Input 0, rr=1 favours Input 1); after a successful contended transfer the pointer flips, so repeated contention does not permanently favour one input. Packet locking pins the output to whichever input is mid-packet — once a beat with TLAST=0 transfers, the output stays locked to that input until the beat with TLAST=1 transfers, preventing packet interleaving. The ready/valid rule VALID && READY is enforced throughout, so an output that is not ready cannot cause the selected input's beat to be lost, which is what allows the router to handle downstream backpressure correctly.

tb_router.sv exercises seven scenarios: basic routing, concurrent traffic, contention on each output, multi-beat packets, packet locking, and backpressure — checking correct destination routing,

correct data/TLAST transfer, round-robin arbitration, packet locking, backpressure handling, and the absence of unexpected packet loss or duplication.

Scenario	Result
Basic routing	PASS
Concurrent traffic	PASS
Output 0 contention	PASS
Output 1 contention	PASS
Multi-beat packets	PASS
Packet locking	PASS
Backpressure	PASS

PART A RESULT: 17 tests passed, 0 failed. All Part A scenarios completed successfully.

3. Part B — SVA Specification, Rationale, and Broken-Variant Verification

routera_sva.sv (with testbench tb_router_sva.sv) specifies the behavioural requirements as executable checks. For each property: what it protects, what bug would slip past without it, and how a violation shows up.

Property	Protects / would-miss / violation signature
Handshake (VALID && READY)	Protects against a beat being counted as transferred when only VALID is high. Without it, a design could silently drop or fabricate transfers under backpressure. A violation shows up as a beat counted while READY was low the same cycle.

Stability under backpressure	Protects against output data/TLAST changing while VALID=1 and READY=0. Without it, a receiver that is stalled could see a packet silently replaced. A violation shows up as output data or TLAST changing before the transfer completes.
Routing correctness	Protects against a beat reaching an output different from its TDEST. Without it, a design could pass every handshake-timing check while silently misrouting traffic. A violation shows up as TDEST not matching the output the beat appeared on.
Arbitration (round-robin)	Protects against the round-robin pointer failing to determine the winner on contention, or failing to flip afterward. Without it, one input could starve the other under repeated contention — a liveness bug, not a safety bug. A violation shows up as the wrong input winning relative to rr, or rr not flipping after a contested transfer.
Packet locking	Protects against arbitration switching inputs mid-packet, which would interleave two packets into one corrupted stream. Without it, each beat could look locally correct while the packet as a whole is scrambled. A violation shows up as the output's locked owner changing before TLAST.
TLAST / lock release	Protects the boundary condition: the lock must release only after the TLAST beat successfully transfers, not before or on a non-transferring cycle. Without it, locking could release early (allowing interleaving) or never (deadlocking the output to one input). A violation shows up as lock state changing on the wrong cycle relative to a successful TLAST transfer.

A deliberately broken variant was used to confirm the properties actually catch violations, not just pass by default. Injected violations included: changing output data while VALID was high and READY was low; releasing a packet lock before TLAST; selecting the wrong requester during contention; and routing a packet to the wrong output. Each was independently detected by the corresponding property above.

The Part B testbench (tb_router_sva.sv) executed 14 test scenarios covering routing, handshake behaviour, output stability under backpressure, arbitration, packet locking, TLAST propagation, reset behaviour, mixed traffic, and round-robin fairness: 35 checks passed, 1 failed. The failure

was in Test 11 (Reset Behaviour): the testbench reported that outputs became active during reset. The router checker's own core assertions (P1–P6B) nonetheless reported CORE CHECKS: PASS, since the reset-behaviour check is part of the testbench harness rather than the P1–P6B property set. Fairness: Output 1 PASS (4 contention wins for each input); Output 0 NOT DEMONSTRATED (0 recorded contention wins for either input — a coverage gap, not an observed violation).

Category	Result
Handshake	PASS
Stability	PASS
Routing	PASS
Arbitration	PASS
Packet-lock	PASS
TLAST	PASS
Reset behaviour	FAIL — outputs active during reset
Broken-design detection	PASS

PART B RESULT: core SVA checks pass (CORE CHECKS: PASS), but the overall testbench contains one reset-behaviour failure (35/36 checks passed). This is reported explicitly rather than being classified as an overall clean pass.

4. Part C — AnvilHDL Implementation and Equivalence

The router was reimplemented in Anvil (router.anvil), generating SystemVerilog RTL (router_generated.anvil.sv). Architecture: each input feeds a per-input router process, which forwards requests to per-output arbiter processes over synchronous channels; each output arbiter implements arbitration, round-robin state, packet locking, owner tracking, and TLAST-based lock release. Each beat is encoded as a 34-bit value {TLAST, TDEST, TDATA[31:0]}. Compiled with AnvilHDL 0.1.0 (opam, pinned to commit d138cabedbf3b65c08249ce6a55cb90dad959da).

Compilation: `anvil -just-check router.anvil` (PASS) → `anvil -o router_generated router.anvil` (PASS) → `verilator --lint-only router_generated.anvil.sv` (PASS, Verilator 5.050).

4.1 Equivalence Test Suite

The Part C test suite (tb_partc.sv) was derived from the behavioural properties specified in Part B, connecting each property to observable test coverage:

The same logical stimulus was applied to both the SystemVerilog reference and the generated Anvil implementation; their output streams were compared for beat count, payload, packet termination, and ordering.

Part B property	Part C verification
Handshake correctness	Basic routing and transfer tests
Stability under backpressure	Backpressure tests
Routing correctness	Directed destination tests
Arbitration correctness	Contention and round-robin tests
Packet locking	Multi-beat packet tests
TLAST / packet completion	Multi-beat and packet-locking tests
Fairness / progress	Repeated contention tests

4.2 Final Build Result

A fresh build was run from the final, committed Part C source and generated RTL (router.anvil → router_generated.anvil.sv → tb_partc.sv). The complete test suite simulated 305 cycles. Both implementations produced identical output beat counts: Output 0 — SystemVerilog 14 beats, Anvil 14 beats; Output 1 — SystemVerilog 14 beats, Anvil 14 beats.

Two functional differences were detected, both on Output 0, both under contention:

Output 0 beat 4: SV = e0000001 Anvil = e0000002

Output 0 beat 5: SV = e0000002 Anvil = e0000001

The two implementations delivered the same number of beats with the same payload set, but two contending beats appeared in the opposite order on Output 0.

PART C RESULT: strict ordering equivalence was not achieved. The implementation matched the reference in output beat counts and in all other tested routing, packet, backpressure, and locking scenarios; the remaining discrepancy is an arbitration-order swap under contention. This result is the final, confirmed Part C measurement and replaces both earlier reported results (243 cycles/1 error and 252 cycles/227 mismatches), which were intermediate repository states.

Semantic gaps between SVA/SystemVerilog and AnvilHDL, condensed:

- Handshake: SVA asserts directly on VALID/READY; Anvil expresses the same communication via channel send/recv, so properties don't translate syntactically one-to-one.
- Temporal assertions: SVA's multi-cycle stability properties (e.g. "data must hold while stalled") have no direct Anvil equivalent; these are checked indirectly via Anvil's synchronous execution model and the external equivalence testbench.
- Backpressure: downstream READY affects Anvil's acknowledgement through channels/buffers rather than directly, so the exact cycle of acknowledgement can differ without affecting eventual correctness.
- Arbitration scheduling: process/channel scheduling in Anvil means the arbiter can observe two simultaneously-intended contenders at different logical times than the SystemVerilog reference, which is the direct mechanism behind the Output 0 ordering swap above.
- Packet locking: implemented explicitly with lock/owner state in SystemVerilog (checkable directly with SVA) versus inside the Anvil arbiter process (checked via the equivalence testbench instead).

5. Anvil Investigation Observations (brief §1.4)

5.1 A design Anvil's checks rejected

Porting the router surfaced repeated compiler rejections: channel-syntax errors, message-direction mismatches, zero-delay recursion errors (a same-cycle dependency where a signal's definition depended on itself — exactly the "stable value" hazard Anvil's type system is built to catch), communication/thread-structure issues, endpoint/ready-syntax issues, and structural multiple-writer conflicts. These drove a redesign toward dedicated internal routing channels and explicit per-output arbitration state.

Minimal reduced example:

```
proc test() {  
    reg valid0 : logic;  
    set valid0 := 1'b0;  
    loop { cycle 1 }}
```

The compiler rejects a bare set placed directly in the procedure body rather than inside an executable control-flow construct — state updates must be located where the compiler can establish their timing relative to a cycle boundary.

The compiler-rejection example was observed during an earlier Anvil implementation experiment (router_equiv.anvil) and is retained as evidence of an implementation limitation encountered during development. It is not a line-level reference to the final router_equiv.anvil in its current, restructured form — that file has since been reorganised so assignments occur inside supported execution constructs, and the specific line number from the original error no longer applies to it.

5.2 A design Anvil accepts that is nonetheless wrong

This is the kind of gap the Part C equivalence run above is built to catch: Anvil's type system guarantees timing safety (no unstable combinational value, no illegal concurrent write) but does not by itself guarantee transaction-level or ordering equivalence with an independently written reference. The Output 0 arbitration-order swap in §4.2 is precisely this case: Anvil accepted a design that is structurally timing-safe, while the equivalence test against the SVA-derived reference behaviour caught an incorrect observable result (two contending beats delivered in swapped order).

5.3 A construct traced through the compiler

The Anvil statement `send out0.beat(*packet0) >>` compiles to three cooperating pieces of generated SystemVerilog: an output-valid signal derived from the compiler's internal thread-event state, a data-selection mux keyed on which event fired, and an acknowledgement-gated event-advance representing completion of the channel handshake — e.g. `assign _out0_beat_valid = (EVENTS0[7].event_current || _thread_0_event_syncstate_8_q) || (EVENTS0[15].event_current || _thread_0_event_syncstate_16_q);`. This event/sync-state machinery is what the type system reasons over to guarantee timing safety.

6. Tools, Reproducibility, and Limitations

Developed on macOS with AnvilHDL 0.1.0 (opam, pinned to commit d138cabedbf3b65c08249ce6a55cb90dad959da), Icarus Verilog, and Verilator 5.050. Yosys/SymbiYosys were not available locally, so final local validation used Anvil, Icarus Verilog, and Verilator as the recorded equivalent toolchain.

One command reproduces the full build and all three parts, using the final committed files (router.anvil, router_generated.anvil.sv, tb_partc.sv):

```
anvil -just-check router.anvil && \  
anvil -o router_generated router.anvil && \  
verilator --lint-only router_generated.anvil.sv && \  
iverilog -g2012 -o parta_sim router.sv tb_router.sv && \  
vvp parta_sim && \  
iverilog -g2012 -o partb_sim router.sv routera_sva.sv  
tb_router_sva.sv && \  
vvp partb_sim && \  
iverilog -g2012 -o partc_sim router.sv router_generated.anvil.sv  
tb_partc.sv && \  
vvp partc_sim
```

This command was run fresh against the final committed source and produced the 305-cycle, 14-beats-per-output, 2-ordering-difference result reported in §4.2. It should be recorded as the single reproduction command in the README.

Push to GitHub main confirmed: the files (router.anvil, router_generated.anvil.sv, tb_partc.sv, router.sv, routera_sva.sv, tb_router.sv, tb_router_sva.sv) and results reported throughout this report are live on the repository's main branch.

7. Conclusion and Provenance

The project delivers a working SystemVerilog reference router, an SVA verification layer with per-property rationale and a verified broken-variant check, and an AnvilHDL reimplementaion that compiles, lints, and matches the SystemVerilog reference in output beat counts and in all tested routing, packet, backpressure, and locking scenarios, with one confirmed arbitration-order discrepancy under contention. All results reported above were produced by a fresh build and run against the final committed source; no claim of strict ordering equivalence is made, since that discrepancy remains open and is documented rather than hidden.

What I'd change about Anvil: the project highlighted a distinction between timing safety and behavioural equivalence. Anvil's communication model provides timing-safe synchronous communication, but this does not automatically guarantee cycle-by-cycle or ordering equivalence with an independently written ready/valid SystemVerilog implementation — establishing that required explicit, separate reasoning about transaction acceptance, backpressure, arbitration state, packet locking, and ordering. A useful improvement would be higher-level Anvil support for expressing and verifying externally observable transaction semantics (in particular contention ordering), which would make equivalence against conventional ready/valid RTL easier to establish and check statically rather than only by external equivalence testing.

External reference designs: no RTL source code was directly reused from RaveNoC, verilog-axis, the ZipCPU skidbuffer, or another external router implementation. The SystemVerilog router and Anvil implementation were developed specifically for this assignment; AnvilHDL documentation and the Anvil research paper were consulted for language semantics and compiler usage.

Provenance: AI assistance was used for understanding AnvilHDL syntax and communication semantics, debugging compilation issues, developing and reviewing SystemVerilog/SVA code and the equivalence testbench, interpreting simulation results, and preparing documentation. The implementation and testing were completed by the author, with AI used as a supporting tool throughout development.